search (root, element)

→ if (root == empty) then

→ return false

→ if (root's data == element) then

→ return true;

→ if (element < root's data) then

→ return search (root's left child, element);

→ return search (root's right child, element);

↓ Removed tail recursion

Search (root, element)

→ while (root != empty)

→ if (root's data == element) then

→ return true;

→ if (element < root's data) then

→ root = root's left child

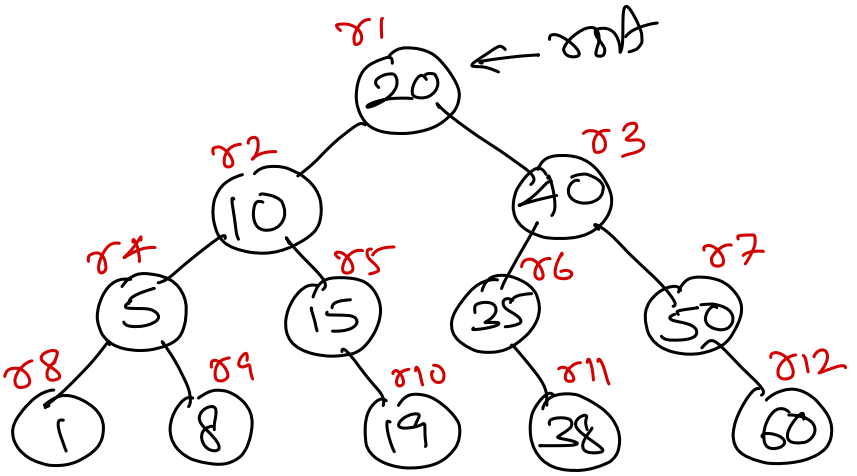
else

→ root = root's right child

→ return false;

Search (element)
- Set current to root.
- while (current is not empty) do
  - if (current node's data = element) then
    - Element found.
    - Stop.
  - if (element < current node's data) then
    - Move current to current's left child.

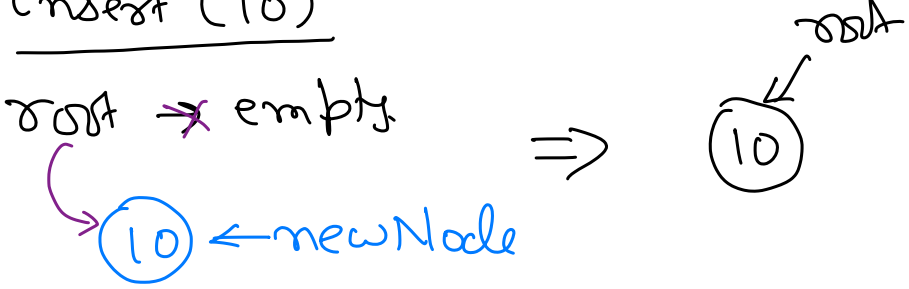
  Else
    - Move current to current's right child.
- Element NOT found.
- Stop



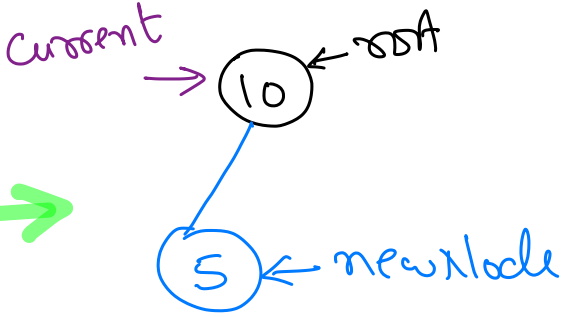
```java
public void treeOperation() {
    Stack<Node> stack = new Stack<>();
    Node current = root;

    while (current != null || !stack.isEmpty()) {
        while (current != null) {
            if (current.right != null) {
                stack.push(current.right);
            }
            stack.push(current);
            current = current.left;
        }

        current = stack.pop();

        if (!stack.isEmpty() && current.right == stack.peek()) {
            stack.pop();
            stack.push(current);
            current = current.right;
        } else {
            System.out.print(current.data + " ");
            current = null;
        }
    }
    System.out.println("");
}
```

Create a BST

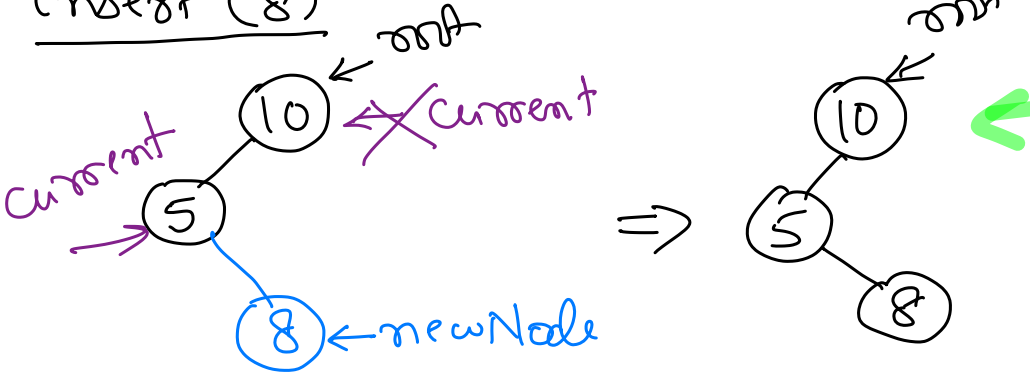
insert (10)



insert (5)



insert (8)

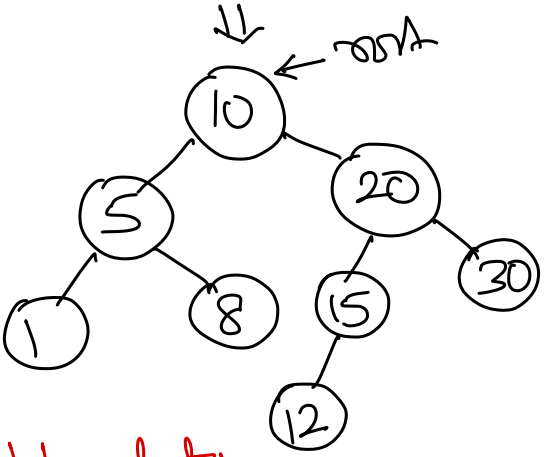


insert: 20, 15, 30, 1, 12

Insert (element)
- Make space for the new element, say newNode.
- Store the element in newNode's data.
- Set newNode's left and right child to empty.

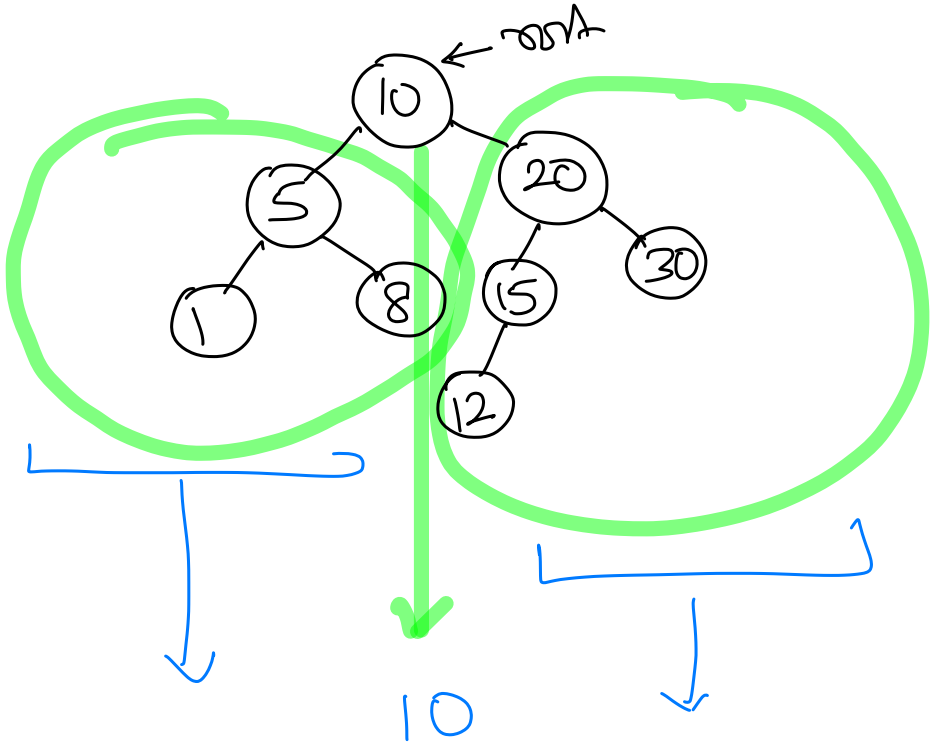
- if (tree is empty) then // root is empty
  - Make newNode as the root node of tree.
  - Stop.

- Set current to the root node.
- while (current is not empty) do
  - if (element < current node's data) then
    - if (current's left child is empty) then
      - Set newNode as left child of current.
      - Stop.
    - Move current to current's left child.
  - Else
    - if (current's right child is empty) then
      - Set newNode as right child of current.
      - Stop.
    - Move current to current's right child.
- Stop



if (current's data == element) then
→ Stop.

do not allow duplicate values.


10
5
20
1
8
15
30
12
10

Time complexity of BST operations

Search

Iteration #	# of nodes
1	n
2	$\frac{n}{2}$
3	$\frac{n}{4}$
⋮	⋮
?	1

? $= \log_2 n$

$10000 \xrightarrow{\text{div by } 10}$ 1 (4 times)

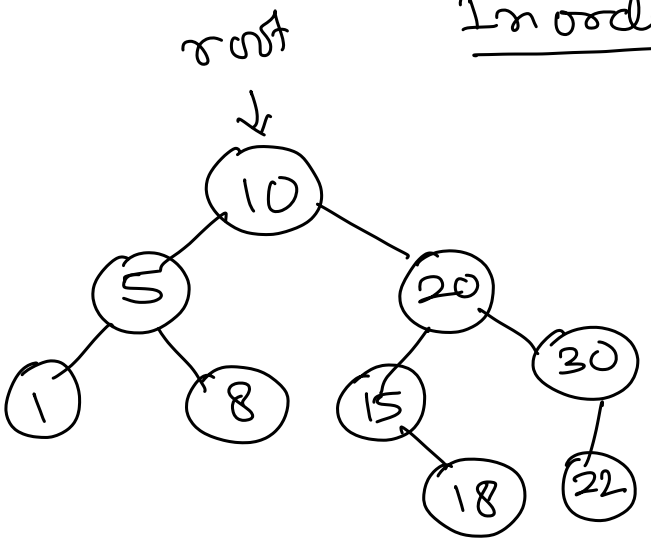
$\log_{10} 10000 = 4$

$10^4 = 10000$

Search time complexity $= O(\log_2 n)$

Insert $= O(\log_2 n)$

logarithmic



Inorder Successor → Node that gets processed after a node during inorder traversal.

Inorder Traversal

1 5 8 10 15 18 20 22 30

Steps to find inorder successor (ios)

1. Set ios to right child of node.
2. while (ios has left child)
   → Move ios to ios left child.

Work when inorder successor is in right subtree of node.

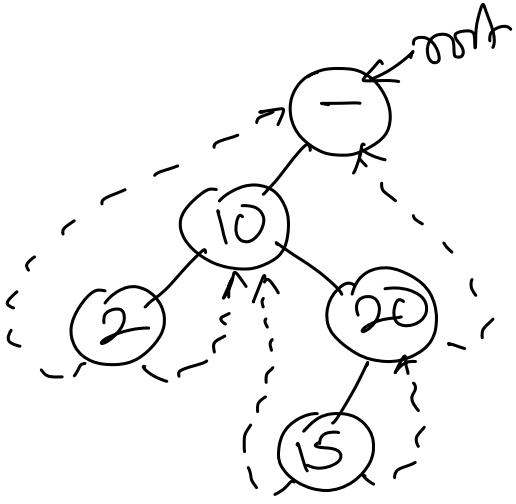
Inorder predecessor → Node that gets processed before a node during inorder traversal.

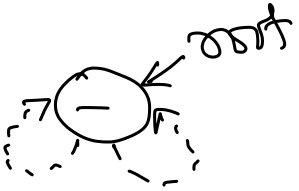
Steps to find inorder predecessor (iop)

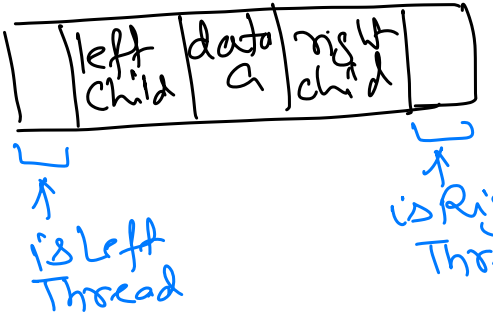
1. Set iop to left child of node.
2. while (iop has a right child)
   → Move iop to iop right child.

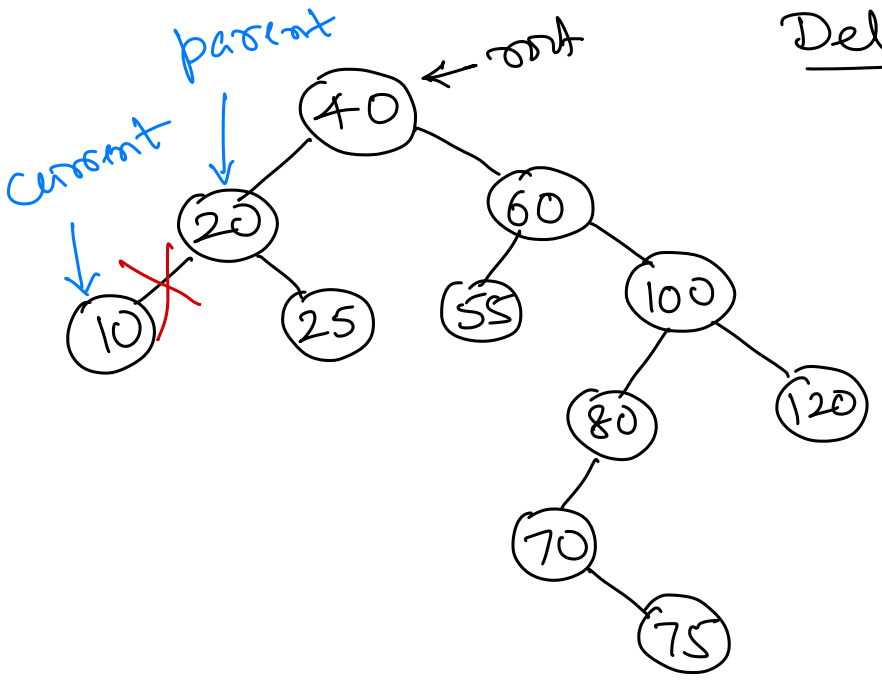
Work when inorder predecessor is in left subtree of node.

Threaded Binary Tree







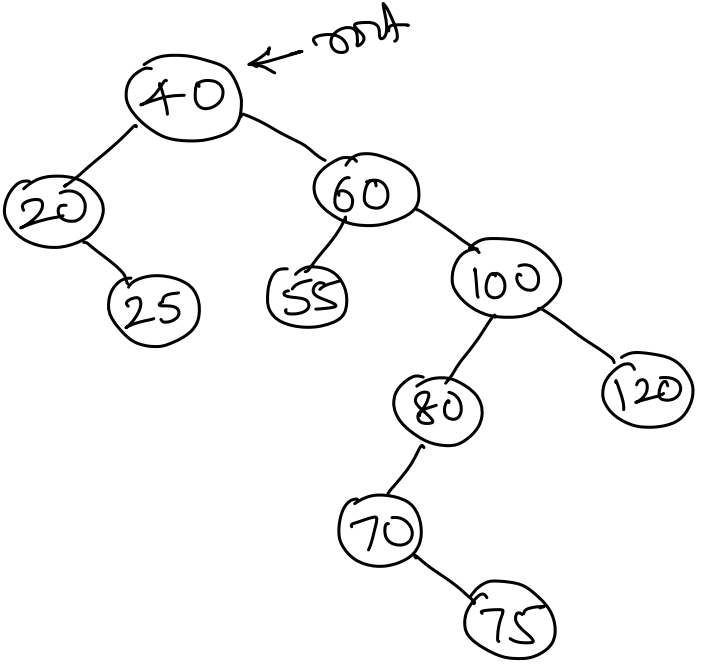


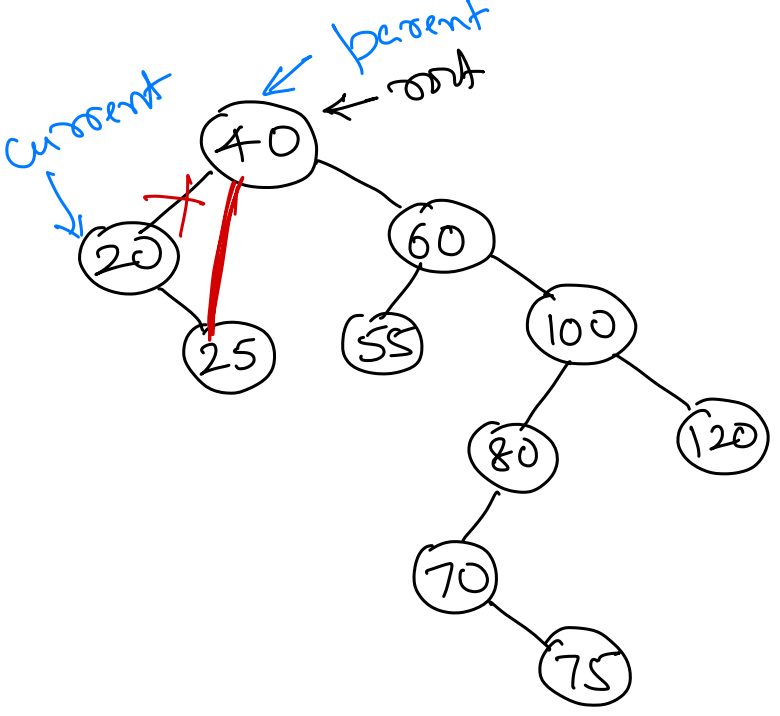
Delete in BST

delete (10)

① when current node (node to be deleted) is a leaf node.

→ Remove current as a child of its parent.

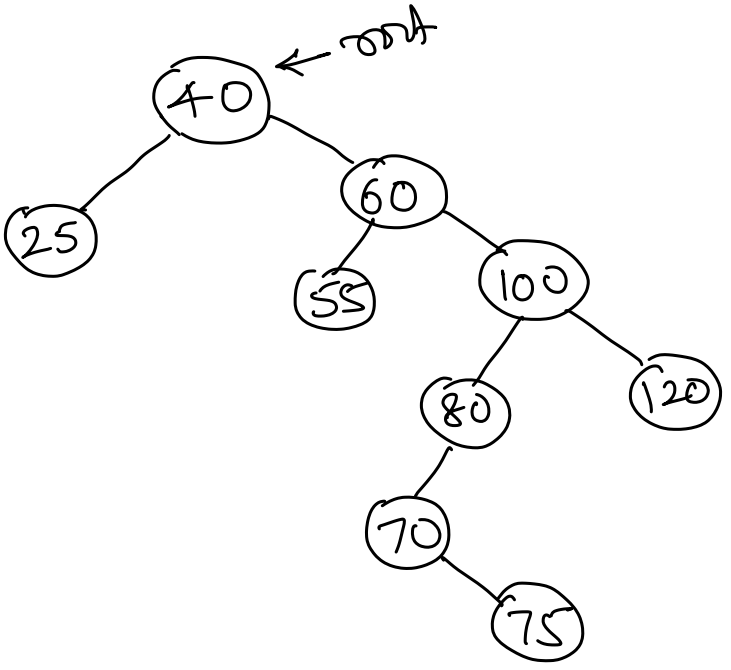

root
40
20
60
25
55
100
80
120
70
75

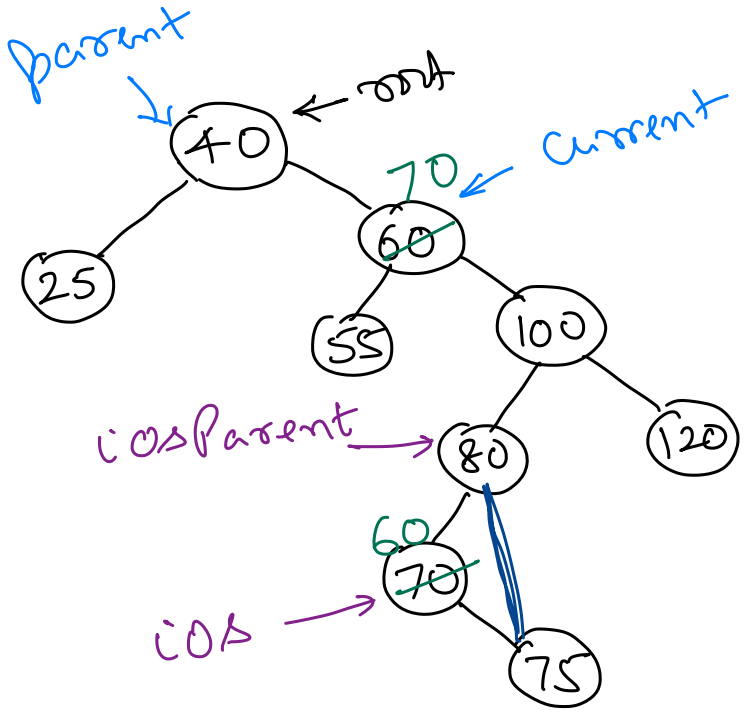


delete (20)

(2) When current node has one children.

→ current node's only child replaces current as child of its parent.


root
40
25
60
55
100
80
120
70
75



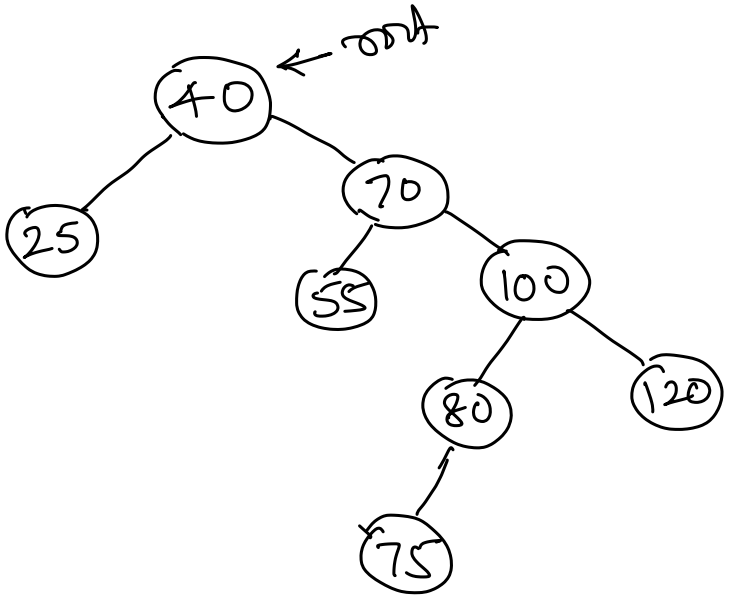
delete (60)

(3) when current has both child nodes present.

→ Find inorder successor/ predecessor of current.

→ Swap values of current & inorder successor / predecessor

→ Delete inorder successor/ predecessor node



root → empty

delete(10)

10 ← current

parent → empty

⇓

root → empty.

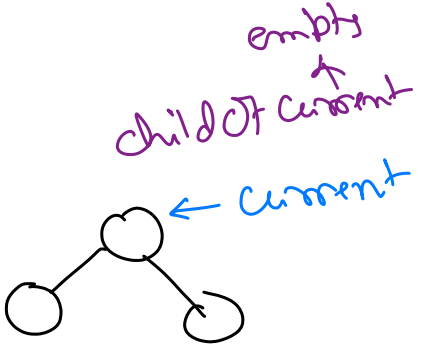
Delete(element)
// Find the node to be deleted.
- Set parent to empty
- Set current to root node.
- while (current is not empty) do
  - if (element = current node's data) then
    // Element found.
    - End the traversal.
  - Set parent to current.
  - if (element < current node's data) then
    - Move current to the current node's left child.
  Else
    - Move current to current node's right child.

// Is an element found?
- if (current is empty) then
  // Element not found in tree.
  - Stop

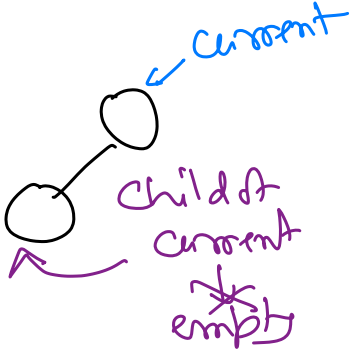
// Deleting leaf node?
- if (current is leaf node) then // Leaf node => both child are empty
  // Are we deleting root node? => Deleting the only node in the tree.
  - if (current is root node) then
    - Set root to empty.
    - Release memory for the current node. // Not required in JAVA.
    - Stop.

Time complexity = ?

// Delete current node, child of parent. But, which child?
- if (current is left child of parent) then
  - Set left child of parent to empty.
Else
  - Set the right child of the parent to empty.
- Release memory for the current node. // Not required in JAVA.
- Stop



// Deleting node with only one child?
- Set childOfCurrent to empty.
- If (current node's left child is empty) then // current has only right child.
  - Set childOfCurrent to current's right child.
- if (current node's right child is empty) then // current has only left child.
  - Set childOfCurrent to the current's left child.
- if (childOfCurrent is not empty) then // Current has only one child.
  // Set the only child of the current as the child of its parent.
  - if (current is left child of parent) then
    - Set childOfCurrent as left child of parent.
  Else
    - Set childOfCurrent as the right child of the parent.
  - Release memory for the current node. // Not required in JAVA.
  - Stop.



// Deleting node with two children.
// Find inorder successor of the current.
- Set inOrderSuccessorParent to current.

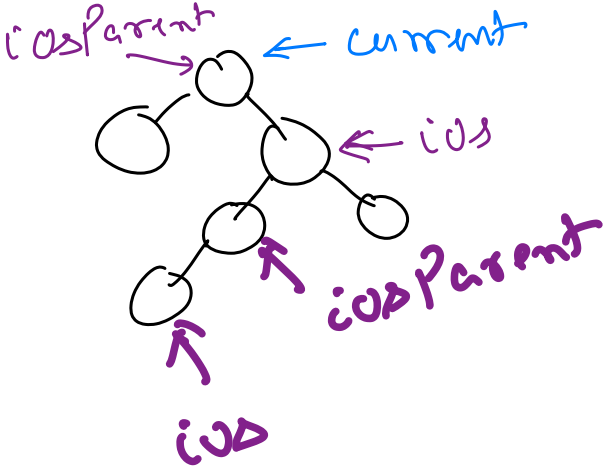
- Set inOrderSuccessor to the current node's right child.
- while (inOrderSuccessor has left child) do
  - Set inOrderSuccessorParent to inOrderSuccessor.
  - Move inOrderSuccessor to the left child of inOrderSuccessor.
- Swap data of current and inOrderSuccessor node.

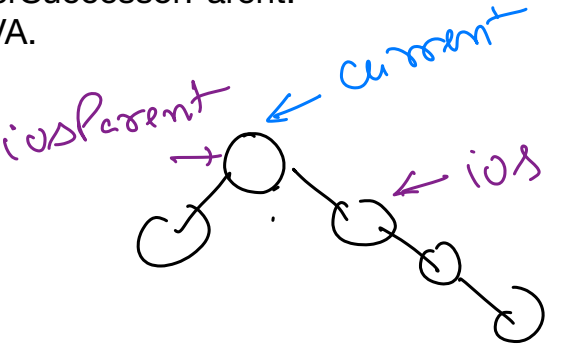
// Delete inorder successor node.

// Inorder successor has max one child. => It will only be the right child.

- if (inOrderSuccessor is left child of inOrderSuccessorParent) then
  - Set right child of inOrderSuccessor as left child of inOrderSuccessorParent.

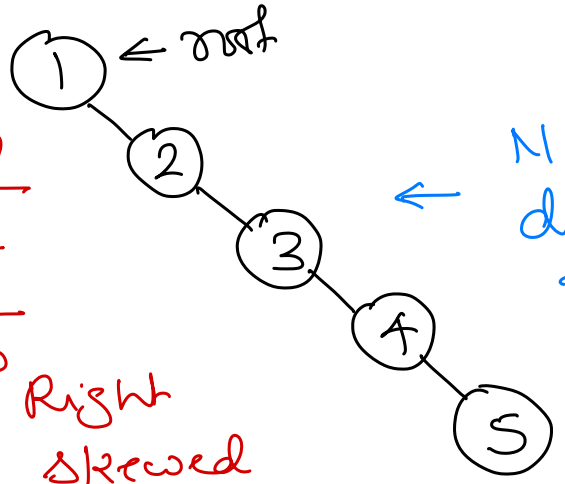
Else
  - Set the right child of inOrderSuccessor as the right child of inOrderSuccessorParent.
- Release memory for inOrderSuccessor node. // Not required in JAVA.
- Stop





Problem with BST

Insert → 1 2 3 4 5



Skewed Binary Tree

Left Skewed

Each node has only left child

Right Skewed

Each node has only right child

← Nodes are NOT evenly distributed between left and right subtrees.

Seach in Skewed BST

Iteration #	→	# of nodes
1	→	n
2	→	n-1
3	→	n-2
⋮	⋮	⋮
?	→	1

Time complexity = $O(n)$

How to solve the problem?

→ Use balanced BST / self adjusting BST

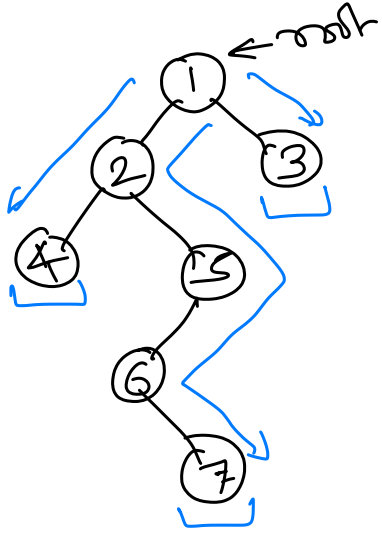
⇓

nodes are evenly distributed after inserting in a BST.

e.g. 2-3 Tree, Red-Black Tree, AVL Tree.

Height of a node in a tree

Number of edges on the longest path from that node to a leaf node.



root → empty

height of root = 0

root → 5

height = 0

1 → 3

= 1 edge

1 → 2 → 4

= 2 edges

1 → 2 → 5 → 6 → 7

= 4 edges.

$$\text{Height}(\text{root}) = \begin{cases} 0, & \text{if root is empty.} \\ 0, & \text{if root is leaf node} \\ 1 + \text{MAX}(\text{height}(\text{root's left child}), \\ \quad \text{height}(\text{root's right child})), \\ \quad \text{otherwise.} \end{cases}$$

AVL Tree

Height balanced binary search tree.

Each node has a balance factor between -1 and +1.

Balance factor (B.F.) = $h_L - h_R$

h_L → height of left subtree

h_R → height of right subtree.